

To Create Intelligent Adaptive Game Opponent by Using Monte-Carlo for Tree Search

Jiajian Yang, Yuan Gao, Suojun He, Xiao Liu, Yiwen Fu, Yang Chen, Donglin Ji
International School, Beijing University of Posts and Telecommunications, Beijing, China, 100876
 yangjeep@gmail.com

Abstract

Adaptive Game AI improves adaptability of opponent AI as well as the challenge level of the game play; as a result the entertainment of game is augmented. The most updated algorithm of MCT (Monte-Carlo for Trees) which perform excellent in computer go can be used to achieve excellent result to control non-player characters (NPCs) in video games. In this paper, the prey and predator game genre of Dead End is used as a test-bed, the basic principle of MCT is presented, and the effectiveness of its application to game AI development is demonstrated. Furthermore, by using the construction of ANN (Artificial Neural Network) trained by the data collected from Monte-Carlo Method, the validation of effectiveness and efficiency of the approach is presented. Finally, an approach of combine both the two techniques is discussed to create the intelligent adaptive game opponent^[9].

1. Introduction

Adaptive Game AI improves adaptability of opponent AI as well as the challenge level of the gameplay; as a result the entertainment of game is promoted. Opponent game AI is usually implemented by scripted rules in video games, but the most updated algorithm of Monte-Carlo which perform excellent in computer go can also be used to achieve excellent result to control non-player characters (NPCs) in video games. Monte-Carlo-implemented opponent can achieve good adaptability which contributes well to the entertainment of game as it dynamically adjusts to different gamers' possibility versatile of skill.

2. Description of game Dead-End

Dead End is a two-dimensional multi-agent predator/prey game. It receives some recent attention

in the AI research, especially in the domain of Machine Learning. The game's fundamental concept is inspired by previous work of Yannakakis^[1]. In this paper minor modification is applied to the Dead-End game to facilitate the experiment. As it shown in the figure, the blue oval represents the Cat and the yellow ones are the Dogs. The green boxes are the exits for Cats.

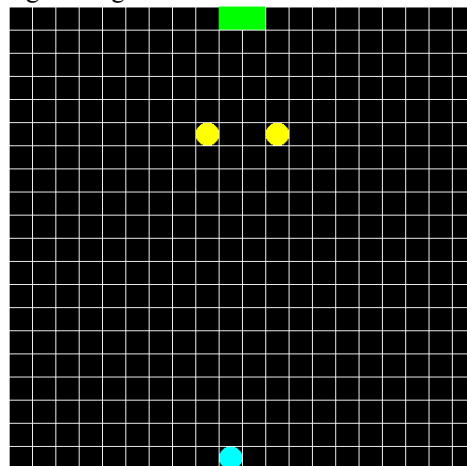


Figure 1 Dead-End game

The aim of the Cat or the Player, starting from a stationary chosen position at the bottom of the stage, is to reach the Exit by avoiding the 2 Dogs which are also starting from a stationary chosen position, or to survive for a predetermined large period of time, in the test, 20 simulation steps. On the other hand, Dogs are aiming to catch the Cat to prevent it reach the exit within that period of time. Cat and Dogs are initially placed in the game field so that there is a suitably large distance between them. Then, the following occur at each simulation step of the game: both Cat and Dogs gather information from their environment and take a movement decision of up, down, left or right. The game is over if a Player succeeds in arriving at the Exit within predetermined simulation steps, which is 20 in the test-bed, this event is described as a win. On the other hand, if a Dog manages to catch a Player, this

event defines a kill. If there is neither a Player win nor a kill within the limited steps, then the outcome of the game is a draw.

Conceptually, the 2 Dogs are designed to be slower than the Cat so that the game is fairer to play. The Cat moves at two times the Dogs' speed, so it is impossible for a single Dog to complete the task of killing it. Therefore the effective way to catch the Cat is to hunt cooperatively for the group of Dogs. It is worth mentioning that one of the Dogs' properties is permeability: two or more Dogs can simultaneously occupy the same position on the game field.

3. Background of Monte-Carlo Method

Monte-Carlo is a learning method for estimating value functions and discovering optimal policies. Monte Carlo methods are a class of computational algorithms that rely on repeated random sampling to compute their results. Monte Carlo methods are often used when simulating physical and mathematical systems. Because of their reliance on repeated computation and random or pseudo-random numbers, Monte Carlo methods are most suited to calculation by a computer. Monte Carlo methods tend to be used when it is infeasible or impossible to compute an exact result with a deterministic algorithm^[6].

4. Apply Monte-Carlo algorithm to improve the adaptability of the opponent AI in Dead-End

This section shows the process of applying Monte-Carlo algorithm to the test bed in order to prove the adaptability of Dead-End. Section 4.1 describes the general application of Monte-Carlo Method to the game. Section 4.2 illustrates the relation between the performance of Monte-Carlo and its computation time limit. Then, by using the data collected in the experiments before, we construct several Artificial Neural Networks towards different strategies of player. Section 4.3 demonstrated the process of building the ANN (Artificial Neural Network). After the construction of ANN, it is able to prove the matching between ANN and particular player strategies. Section 4.4 showed the cross-experiment of the proof. Finally, in section 4.5, by comparing the performance and resource consumption we are able to recognize the effectiveness and efficiency of the whole process in the construction of intelligent adaptive game opponent.

4.1. Application of Monte-Carlo to Dead-End

The application of Monte-Carlo in Dead-End is based on the hypothesis that the method to control each Dog in game board is actually determined by a choice of legal move. The possible legal moves could be in the direction of Up, Down, Right, and Left. Therefore a Dog is free to choose a direction to go as long as there is no boundary on that direction which is not allowed to go.

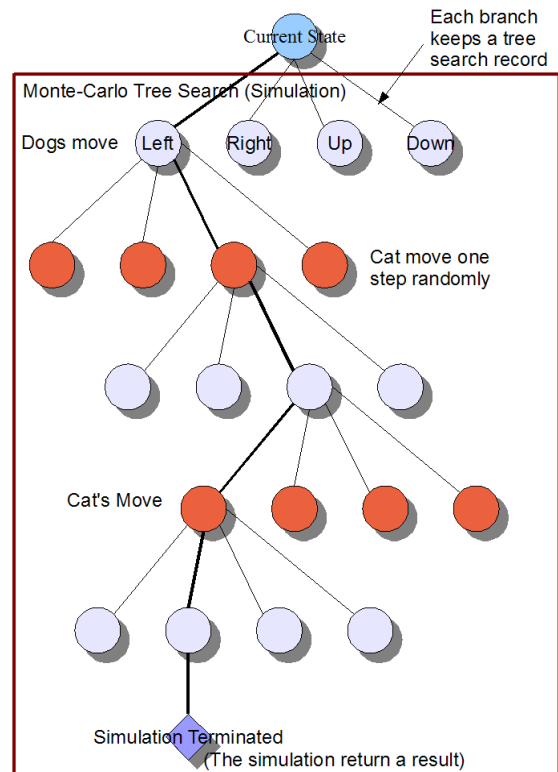


Figure 2 The Monte-Carlo Tree Search in Dead-End

At every time-step the Monte-Carlo algorithm builds a search tree consisted of possible legal moves, this ensures that no good moves would be omitted. Thus a tree with four parent nodes is built and consequently Monte-Carlo Tree Search is applied to launch the simulation. Once in the simulation is terminated with a result the branch of the top level which contains four nodes will record the result correspondingly. Therefore at the end the node on the top of the decision tree with the highest winning rate will be picked and the Dog will move according to the direction the node represents.

4.2. Performance of Monte-Carlo Method towards time limit in Dead-End

Considering Monte-Carlo is computation intelligence approach, its performance relies on the

time limit of the computation. This makes the performance curve of Monte-Carlo Method important in the application. The experiment is proposed to let the Dogs (NPC) fight against the Cat with three different strategies. By adjusting the computation time limit of Dogs, discrete points which indicate the relation between the computation time and the performance could be spotted. Then by applying the regression algorithms on the discrete points the function of the curve could be drawn. The desired computation time limit is under 500 milliseconds, since that will not make users uncomfortable with the computation. However, to find out the curve some points greater than 500 has been tested too.

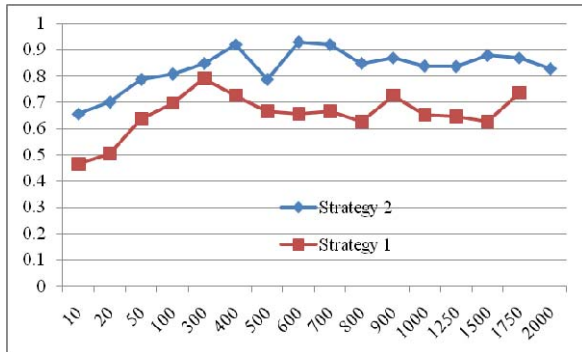


Figure 3 Performance of Monte-Carlo vs Two distinguishable strategies towards time limit

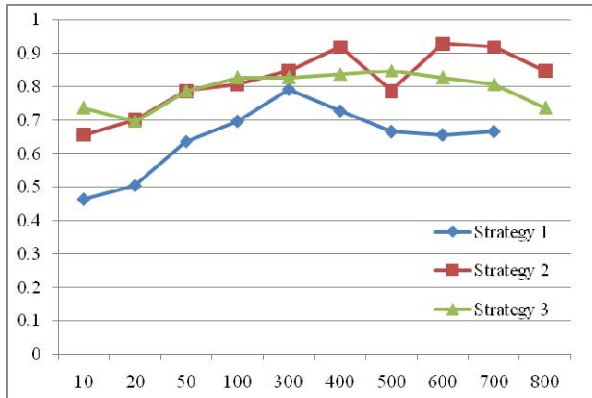


Figure 4 Performance of Monte-Carlo towards 3 Strategies towards time limit

From the curve we can see that at a limited interval the performance of Monte-Carlo increases with respect to the increase of computation time. However, after a threshold value, the performance decreases down to a stable state and keep a stable value. This phenomenon is due to the nature of Monte-Carlo Tree Search. When the computation increases from 0 to a particular interval, the width of the tree search becomes greater which causes the performance better and better.

4.3. Construct Artificial Neural Network from Monte-Carlo

Since Monte-Carlo method takes effects together with great time consumption, it cannot be directly applied to the current video game. Otherwise it will no doubt bring annoying latency which is horrible for video game users. Therefore a more efficient algorithm should be used to substitute Monte-Carlo Method while it has to imitate the performance of Monte-Carlo Method. Artificial Neural Network (ANN) could achieve this goal by applying good training and optimisms. In the experiment we use the widely used model of 3-Layered Feed-Forward Artificial Neural Network.

The experiments aiming at validate the effectiveness of applying ANN to the game is proposed as follows. Firstly, we collect the data of every move of the Dog and Cat in the past game. There are 28 attributes in total are collected in every step, and these are also the inputs of the Neural Network.

The attributes are:

catToDog1x, catToDog1y, catToDog2x,
catToDog2y, catToDog1, catToDog2, catToExitX,
catToExitY, catDog1Angle, catDog2Angle,
catToLeft, catToRight, catToTop, catToBottom,
dog1ToLeft, dog1ToRight, dog1ToTop,
dog1ToBottom, dog2ToLeft, dog2ToRight,
dog2ToTop, dog2ToBottom, dogInnterDist,
dog1ToExitX, dog1ToExitY, dog2ToExitX,
dog2ToExitY, turn

These are thought enough to describe the situation of the game board and enough for the Neural Network to make the decision. In the stage of processing the data, we filtered the data by deleting the data from the game that the Dog lost the game. In other words, we save only the data from the game in which the Dog won (might be better with bad data). We assume that these data are "good". After that, we split the data into two different parts, one for training the ANN, the other part for validating the weights and bias of the ANN. Consequently, three ANNs are built with respect to the three strategies of Cat.

After the construction of the three ANN, we replace the "Brain" of the Dog. Each Dog has an ANN with 28 inputs and 4 outputs within 18 hidden neural nodes in the hidden layer of the ANN. The data is transferred into the input node dynamically before every move of the Dog. By the calculation of the ANN, the "Brain" takes the value from the output node and find out which direction is to go next.

To validate the effect of ANN, the Dog equipped with ANN towards the corresponding strategies of Cat is set to fight in the game.

4.4. Cross-Validation Experiment

The drawback of the Neural Network comparing to Monte-Carlo Method is that Neural Network is not capable to fight with unknown strategies. ANN is thought to have learning effect but not universality. Its performance is better when fighting with the strategy where it comes from. Unless a strategy “covers” another, the ANN corresponding to it will not take good effect when fighting with other strategies. We assume we have several ANN which are trained by the data collected by the games that Dogs with Monte-Carlo method fight with Cat with strategies. We call this mapping “ANN from the strategy”. Thus, if no strategy covers the others, there exists a relation that: for an ANN x , its win-lose ratio R towards Cat with different strategies, we call it Strategy y , the R_{x-y} have its maximum only when $x=y$. The cross-validation experiment is set to prove this assumption as follows.

	Cat Strategy 1	Cat Strategy 2	Cat Strategy 3
Dog ANN 1	59.00%	24.00%	20.00%
Dog ANN 2	31.00%	57.00%	70.00%
Dog ANN 3	0.00%	0.00%	100.00%

Table 1 Cross-Validation Experiment of ANN vs different strategies

In the table, we can see that Strategy 2 covered the strategy 3 in a great scale. That dues to the less randomness of Cat Strategy 3. Recall the construction and the mathematical nature of Artificial Neural Network we can easily understand this issue. When building an ANN, we train the ANN to form its weights and bias values. These values are determined by the data in the training set. The training set is set by collecting the data when Dogs using Monte-Carlo Method fighting the Cat under the control of particular strategy. Therefore the range of the data in the training set decides the coverage of the ANN, and consequently affects the performance of the ANN. It could be inferred that the more random a strategy is, the stronger the ANN from it is.

Furthermore, it could be seen that the performance of ANN relies mainly on the correspondence, or mapping of its training set. An ANN performances good when $x=y$ or ANN x covers Strategy y if we have Dogs carrying ANN x fighting with Cat using Strategy y .

By the analysis of the data, we are able to draw the conclusion. First, the application of Artificial Neural Network in the test bed describes that it is possible to replace Monte-Carlo Method partially by ANN. Secondly the drawback of ANN is obvious. It takes effects only when fighting the strategy it covers or it comes from. That means, the task to defeat users with unknown, or, say, uncovered strategies still relies on the computation intelligence, which is Monte-Carlo Method in this thesis. The last but not the least, by adjusting the training set of ANN the dynamic difficult adjustment could be achieved.

4.5. Performance improvement of Neural Network comparing to Monte-Carlo

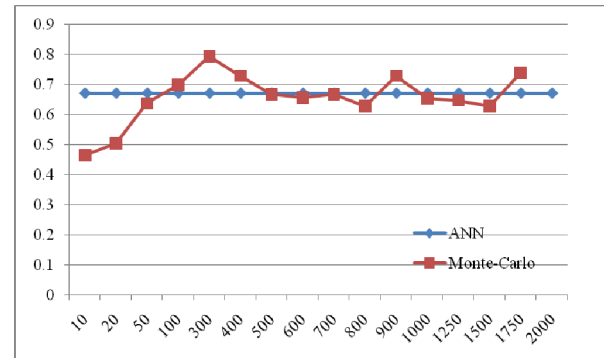


Figure 5 Performance Comparison between Monte-Carlo and ANN fighting with same Strategy

Basically, the computation time consumption of Monte-Carlo Method could be controlled manually by setting the limit of computation. But, it is impossible to determine the performance of it by the same way. Since its randomness and time consumption, its performance cannot be easily adjusted by programmer. However, it is capable to generate “good” data to beat unknown strategies. Comparing to Monte-Carlo, ANN is good at its efficiency and minor time consumption. Even in the toughest environment, ANN will show its absolute advantage comparing to Monte-Carlo method in time consumption. In our experiment, we build 1000 thread running the game board on the computer. The performance of ANN remains stable and has no obvious decrease. However, the Monte-Carlo Method cannot adapt that extreme environment.

As shown in the diagram, ANN is independent of the computation time allocated to it, while Monte-Carlo requires good computation subsystem resources.

5. Conclusion

Monte-Carlo Method is a good algorithm of computation intelligence; it depends on the

computation limit and causes great consumption. It could generate the training data set to build the Neural Network. Monte-Carlo Method in game development could find the optimized solution towards strategies. What's more, it could help to find the solutions to fight against unknown strategies. Artificial Neural Network can be trained by the data collected by Monte-Carlo Method and deal with the known strategies. It has good correspondence towards particular strategies. The process of creating intelligent adaptive game opponent should combine both the Monte-Carlo Method and ANN. Monte-Carlo Method could be used to firstly find out the way to defeat the players and then ANN is trained to play with known players while Monte-Carlo could use idle resources to deal with unknown game players.

6. Acknowledgement

The research is supervised by Professor Suoju He and Mr.Fan Xie from School of Software Engineering Beijing University Of Posts and Telecommunications. The expense of the research is supported by International School Beijing University Of Posts and Telecommunications.

7. References

- [1] Yannakakis, Giorgios N, AI in Computer Games: Generating Interesting Interactive Opponents by the use of Evolutionary Computation. PhD thesis, University of Edinburgh, 2005.
- [2] Georgios N. Yannakakis, John Levine, and John Hallam. An Evolutionary Approach for Interactive Computer Games. In Proceedings of the Congress on Evolutionary Computation (CEC-04), pages 986–993, June 2004.
- [3] Peter Bentley Article, LETTING STONES GO UNTURNED, Feb, 2008.
- [4] Simon M. Lucas, Computational Intelligence and Games: Challenges and Opportunities, International Journal of Automation and Computing, Pages 45-57, January, 2008.
- [5] Sutton R.S. Reinforcement learning. An introduction. 2005
- [6] Monte Carlo method, Wikipedia, Retrieved from http://en.wikipedia.org/wiki/Monte_Carlo_method (03/08/09).
- [8] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In 15th European Conference on Machine Learning (ECML), pages 282–293, 2006.
- [9] Suoju He, Fan Xie, Yi Wang, Jin Meng, Hongtao Chen, Zhiqing Liu, Qiliang Zhu. Game Player Strategy Pattern Recognition and How UCT Algorithms Apply Pre-Knowledge of Player's Strategy to Improve Opponent AI. In the International Conference on Innovation in Software Engineering (ISE'2008), 2008.